

Linux Automation Monitoring, Cleanup, Logging & Auditing

PROBLEM

Managing Linux servers can become difficult when you have to check CPU, memory, disk usage, processes, Docker resources, and EBS volumes separately. I wanted a simple way to bring these everyday monitoring and cleanup tasks into one place.

HOW I DEBUGGED IT

I built a Python script that collects system information, monitors resources, identifies unused processes, checks Docker resources, and gives visibility into what is happening on the server.

THE SOLUTION **Monitor → Detect → Decide → Cleanup → Log → Audit**

I added Auto, Interactive, and Dry-Run modes so cleanup actions can be controlled safely. The script also keeps logs and audit records to make every important action easier to track and review.

Has anyone else faced this problem?

How did you fix it?

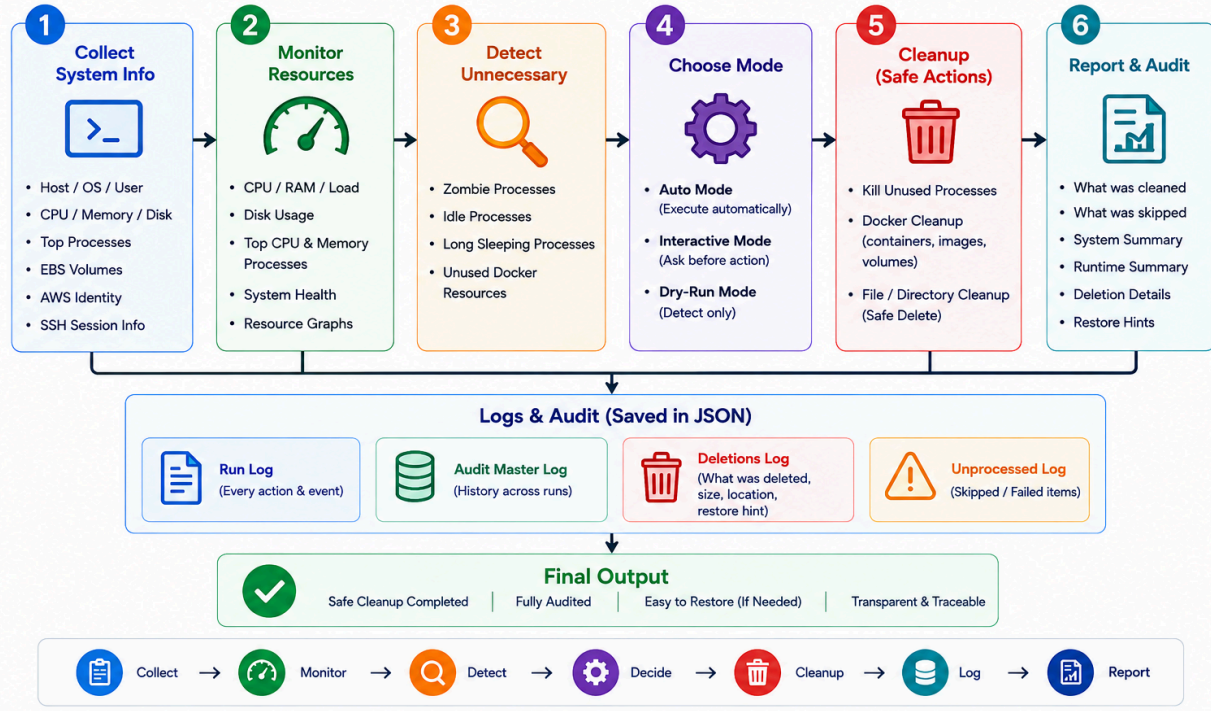
Note: code on the last page, automation set also

Blog: <https://vishnusai.hashnode.dev/linux-monitoring-cleanup-logging-auditing>

<https://vishnusai.hashnode.dev/>

Linux Automation Script

Monitoring → Cleanup → Logging → Audit



Step 1 — Script Overview

Shows the script modes and main features such as monitoring, cleanup, process detection, and logging.

```
ubuntu@ip-172-31-36-199:~$ ./metric.py
/home/ubuntu/.metric.py:41: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  RUN_TS = datetime.datetime.utcnow().strftime("%Y-%m-%dT%H:%M:%S2")
Production Cleanup & Monitoring v2.0

Script Modes:
1. Auto Mode - runs all actions automatically (pre-approved lists only)
2. Interactive Mode [Default] - pauses and asks before every critical action

Features v2.0:
• Dynamic Deletion - files AND directories (recursive), safe + logged
• Docker Mode Selector - choose Auto or Manual Docker cleanup at startup
• Unused Process Mode Selector - independent kill mode per run (Auto/Manual/Dry-Run)
• ASCII Resource Graphs - bar charts for CPU / RAM / disk / Load
• Live Runtime Clock - shows start time + elapsed at every step
• Runner Identity Logging - IP, geo, IAM, Linux user, SSH session
• Unused Process Detection - idle / zombie / sleeping processes
• Unified Log Folder - per-run + master audit + deletion + unprocessed logs
• Restoration Hints - deletion log records size & location for recovery
• Unprocessed Tracker - final report of everything skipped or failed

===== RUNNER IDENTITY =====
Collecting runner identity.../home/ubuntu/.metric.py:78: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use
timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.datetime.utcnow().strftime("%Y-%m-%dT%H:%M:%S2")
done
```

Step 2 — Runner Identity

Collects information about the server, Linux user, IP address, AWS instance, region, and IAM identity.

```
===== RUNNER IDENTITY =====
Collecting runner identity.../home/ubuntu/.metric.py:78: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use
timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
return datetime.datetime.utcnow().strftime("%Y-%m-%dT%H:%M:%SZ")
done

===== RUNNER IDENTITY - Who Is Running This Script? =====

Hostname: ip-172-31-36-199 (ip-172-31-36-199.ec2.internal)
OS/Kernel: Linux-7.0.0-1010-aws-x86_64-with-glibc2.39 - 7.0.0-1010-aws
Started: 2026-08-20T17:29:08Z | Elapsed: 1.9s

----- Network -----
Local IP: 172.31.36.199
Public IP: 107.20.52.148
Location: Ashburn, Virginia, United States | TZ: America/New_York
ISP/Org: Amazon.com, Inc. / AWS EC2 (us-east-1)

----- Linux User -----
Username: ubuntu (uid=1000, euid=1000)
Root?: No
Home/Shell: /home/ubuntu | /bin/bash
Groups: adm(4), cdrom(24), sudo(27), dip(30), lxd(105), ubuntu(1000), docker(988)
SUDO_USER: (not sudo)

----- SSH Session -----
SSH From: 103.90.211.226:57016
TTY/TERM: /dev/pts/0 | xterm
tmux: no

----- AWS Identity -----
Instance: i-00aabb1097c1a58ac5 (m7i-flex.large)
Region/AZ: us-east-1 / us-east-1d
Account: 127696279140
IAM Role: instanceRole
STS ARN: arn:aws:iam::127696279140:user/vishnusai

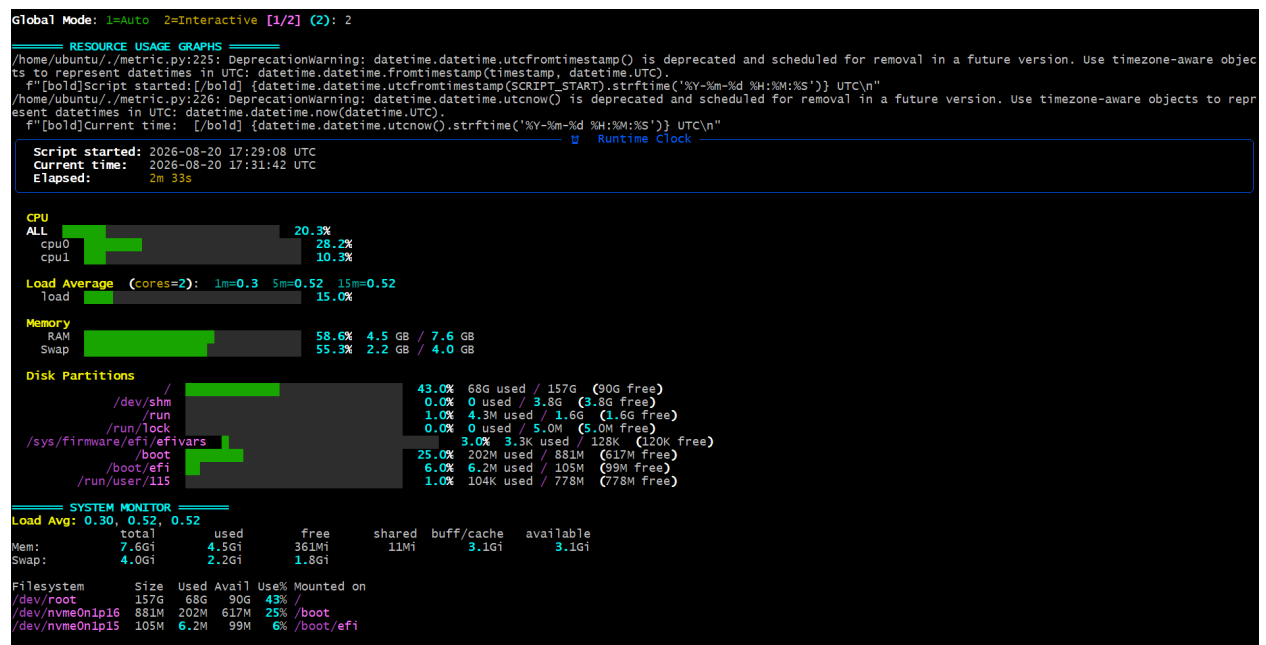
----- Log Folder -----
Log Dir: /home/ubuntu/logs/prod_cleanup
Run Log: cleanup_20260820T172908Z.ndjson
Audit: audit_master.ndjson
Deletions: deletions_20260820T172908Z.json

/home/ubuntu/.metric.py:78: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to repre
sent datetimes in UTC: datetime.datetime.now(datetime.UTC).
return datetime.datetime.utcnow().strftime("%Y-%m-%dT%H:%M:%SZ")

Global Mode: 1=Auto 2=Interactive [1/2] (2): 2
```

Step 3 — Resource Monitoring

Checks CPU, memory, swap, disk usage, and system load to understand the server's current health.



Step 4 — Process Detection

Finds idle, sleeping, and zombie processes and supports Auto, Manual, and Dry-Run modes.

TOP 20 CPU PROCESSES				
PID	PPID	Command	%CPU	%MEM
2895	2387	clickhouse-serv	8.0	6.0
2731	2177	coroot-node-age	3.7	2.2
1080	1	dockerd	1.2	0.7
2959	2398	java	0.9	14.7
164545	1	observability-a	0.8	0.2
874	1	containerd	0.8	0.3
2440	1952	alloy	0.6	1.0
4843	2723	grafana	0.6	2.1
4870	2723	pyroscope	0.6	0.8
2187	1995	mongod	0.6	0.5
4846	2723	toki	0.6	1.1
4849	2723	otelcol-contrib	0.5	0.8
3590	2212	java	0.4	8.8
858	1	glances	0.4	0.1
2595	2147	python3	0.4	1.5
4858	2723	prometheus	0.4	0.8
2726	2290	grafana	0.3	1.5
859	1	node_exporter	0.3	0.2
861	1	teleport	0.2	0.9
4867	2723	tempo	0.2	0.5

TOP 20 MEMORY PROCESSES				
PID	PPID	Command	%CPU	%MEM
2959	2398	java	0.9	14.7
3590	2212	java	0.4	8.8
2895	2387	clickhouse-serv	8.0	6.0
2731	2177	coroot-node-age	3.7	2.2
5904	2846	node	0.1	2.1
4843	2723	grafana	0.6	2.1
2595	2147	python3	0.4	1.5
2726	2290	grafana	0.3	1.5
4846	2723	toki	0.6	1.1
2440	1952	alloy	0.6	1.0
861	1	teleport	0.2	0.9
4870	2723	pyroscope	0.6	0.8
4849	2723	otelcol-contrib	0.5	0.8
4858	2723	prometheus	0.4	0.8
1080	1	dockerd	1.2	0.7
2876	2404	prometheus	0.0	0.6
4867	2723	tempo	0.2	0.5
2187	1995	mongod	0.6	0.5
12094	1	fwupd	0.0	0.4

Step 4 — Process Detection

Finds idle, sleeping, and zombie processes and supports Auto, Manual, and Dry-Run modes. Lists attached EBS volumes with their volume ID, availability zone, size, and current state. Lists attached EBS volumes with their volume ID, availability zone, size, and current state

EBS VOLUMES			
DescribeVolumes			
AZ	ID	Size	State
us-east-1d	vol-0bf0101dd0ade171d	163	in-use
us-east-1d	vol-09414fc075ad862bb	37	in-use

⌘ EBS Volumes completed in 1.2s (total runtime: 2m 34s)

⌘ EBS Volumes completed in 1.2s (total runtime: 2m 34s)

Step 5 — Docker Cleanup

Checks stopped containers, dangling images, and unused Docker volumes for possible cleanup.

Found 294 unused/idle process(es).

Dry-Run Summary

no changes made

Zombies : 1 (would be killed in Auto mode)

Long sleepers : 238 (would be logged as unprocessed)

Idle threads : 55 (would be logged as unprocessed)

Total detected : 294

Unused Process Detection completed in 6.1s (total runtime: 2m 40s)

DOCKER CLEANUP

Docker Cleanup Mode

1. Manual - review and confirm each Docker cleanup step

2. Auto - prune all stopped containers, dangling images, unused volumes automatically

Docker mode [1/2] (1): 1

Stopped Containers

No stopped containers.

Dangling Images

No dangling images.

Unused Volumes

#	Volume Name	Driver
1	buzz-prod_buzz-git-data	local
2	buzz-prod_buzz-minio-data	local
3	buzz-prod_buzz-postgres-data	local
4	buzz-prod_buzz-redis-data	local

[1] Remove all [2] Pick by row [3] Skip [1/2/3] (3): 3

Docker Cleanup completed in 32.3s (total runtime: 3m 13s)

Enter PIDs to kill from CPU list (space-separated, blank=skip) O:

Enter PIDs to kill from Memory list (space-separated, blank=skip) O:

Enter folder path to scan (/):

DISK USAGE (/)

Disk Usage completed in 39.7s (total runtime: 3m 53s)

DYNAMIC FILE / DIRECTORY DELETION

Enter full paths to delete (space-separated, blank=skip) O:

Deletion skipped.

Step 6— Cleanup & Audit Logs

Performs selected cleanup actions and records deleted, skipped, failed, and restore-related information

UNPROCESSED / SKIPPED ITEMS			
295 Unprocessed Item(s)			
#	Type	Detail	
1	dryrun_unused	pid=1	stat=S comm=systemd reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
2	dryrun_unused	pid=2	stat=S comm=kthreadd reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
3	dryrun_unused	pid=3	stat=S comm=pool_workqueue_release reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
4	dryrun_unused	pid=4	stat=I< comm=kworker/R-rcu_gp reason=Idle kernel thread note=Dry-run - not touched
5	dryrun_unused	pid=5	stat=I< comm=kworker/R-sync_wq reason=Idle kernel thread note=Dry-run - not touched
6	dryrun_unused	pid=6	stat=I< comm=kworker/R-kvfree_rcu_reclaim reason=Idle kernel thread note=Dry-run - not touched
7	dryrun_unused	pid=7	stat=I< comm=kworker/R-slab_flushwq reason=Idle kernel thread note=Dry-run - not touched
8	dryrun_unused	pid=8	stat=I< comm=kworker/R-netns reason=Idle kernel thread note=Dry-run - not touched
9	dryrun_unused	pid=10	stat=I< comm=kworker/0:0H-kblockd reason=Idle kernel thread note=Dry-run - not touched
10	dryrun_unused	pid=13	stat=I< comm=kworker/R-mm_percpu_wq reason=Idle kernel thread note=Dry-run - not touched
11	dryrun_unused	pid=14	stat=S comm=ksoftirqd/0 reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
12	dryrun_unused	pid=15	stat=I comm=rcu_sched reason=Idle kernel thread note=Dry-run - not touched
13	dryrun_unused	pid=16	stat=S comm=rcu_exp_gp_kthread_worker/0 reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
14	dryrun_unused	pid=17	stat=S comm=rcu_exp_gp_kthread_worker reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
15	dryrun_unused	pid=18	stat=S comm=migration/0 reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
16	dryrun_unused	pid=19	stat=S comm=kprobe-optimizer reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
17	dryrun_unused	pid=20	stat=S comm=idle_inject/0 reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
18	dryrun_unused	pid=21	stat=S comm=cpuhp/0 reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
19	dryrun_unused	pid=22	stat=S comm=cpuhp/1 reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
20	dryrun_unused	pid=23	stat=S comm=idle_inject/1 reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
21	dryrun_unused	pid=24	stat=S comm=migration/1 reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
22	dryrun_unused	pid=25	stat=S comm=ksoftirqd/1 reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
23	dryrun_unused	pid=27	stat=I< comm=kworker/1:0H-kblockd reason=Idle kernel thread note=Dry-run - not touched
24	dryrun_unused	pid=28	stat=S comm=kdevtmpfs reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
25	dryrun_unused	pid=29	stat=I< comm=kworker/R-inet_frag_wq reason=Idle kernel thread note=Dry-run - not touched
26	dryrun_unused	pid=30	stat=I comm=rcu_tasks_kthread reason=Idle kernel thread note=Dry-run - not touched
27	dryrun_unused	pid=31	stat=I comm=rcu_tasks_rude_kthread reason=Idle kernel thread note=Dry-run - not touched
28	dryrun_unused	pid=32	stat=S comm=kauditd reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
29	dryrun_unused	pid=33	stat=S comm=khungtaskd reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
30	dryrun_unused	pid=34	stat=S comm=oom_reaper reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
31	dryrun_unused	pid=36	stat=I< comm=kworker/R-writeback reason=Idle kernel thread note=Dry-run - not touched
32	dryrun_unused	pid=37	stat=S comm=kcompactd0 reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
33	dryrun_unused	pid=39	stat=SN comm=kmsd reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
34	dryrun_unused	pid=40	stat=SN comm=khugepaged reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
35	dryrun_unused	pid=41	stat=I< comm=kworker/R-kblockd reason=Idle kernel thread note=Dry-run - not touched
36	dryrun_unused	pid=42	stat=I< comm=kworker/R-blkcg_punt_bio reason=Idle kernel thread note=Dry-run - not touched
37	dryrun_unused	pid=43	stat=I< comm=kworker/R-kintegrityd reason=Idle kernel thread note=Dry-run - not touched
38	dryrun_unused	pid=45	stat=S comm=irq/9-acpi reason=Sleeping 515m, 0% CPU note=Dry-run - not touched
39	dryrun_unused	pid=46	stat=I< comm=kworker/R-tpm_dev_wq reason=Idle kernel thread note=Dry-run - not touched
40	dryrun_unused	pid=47	stat=I< comm=kworker/R-ata_sff reason=Idle kernel thread note=Dry-run - not touched
41	dryrun_unused	pid=48	stat=I< comm=kworker/R-md_bitmap reason=Idle kernel thread note=Dry-run - not touched
42	dryrun_unused	pid=49	stat=I< comm=kworker/R-md_11bitmap_io reason=Idle kernel thread note=Dry-run - not touched
43	dryrun_unused	pid=50	stat=I< comm=kworker/R-md_11bitmap_unplug reason=Idle kernel thread note=Dry-run - not touched

Step 7— Final Report

Displays the total runtime, section timing, audit logs, unprocessed items, and final cleanup status.

282dryrun_unusedpid=11521|stat=ss|comm=sshd|reason=Sleeping 506m, 0% CPU | note=Dry-run - not touched

283dryrun_unusedpid=12094|stat=ss|comm=fwupd|reason=Sleeping 506m, 0% CPU | note=Dry-run - not touched

284dryrun_unusedpid=100272|stat=I<|comm=kworker/R-tls-strp|reason=Idle kernel thread | note=Dry-run - not touched

285dryrun_unusedpid=121607|stat=ss|comm=obsagent-intake|reason=Sleeping 229m, 0% CPU | note=Dry-run - not touched

286dryrun_unusedpid=186609|stat=I|comm=kworker/u8:1-events_unbound|reason=Idle kernel thread | note=Dry-run - not touched

287dryrun_unusedpid=192871|stat=I|comm=kworker/0:0-events|reason=Idle kernel thread | note=Dry-run - not touched

288dryrun_unusedpid=197129|stat=I|comm=kworker/u8:5-events_unbound|reason=Idle kernel thread | note=Dry-run - not touched

289dryrun_unusedpid=203490|stat=I|comm=kworker/u8:14-flush-259:0|reason=Idle kernel thread | note=Dry-run - not touched

290dryrun_unusedpid=207209|stat=I|comm=kworker/0:1-rcu_gp|reason=Idle kernel thread | note=Dry-run - not touched

291dryrun_unusedpid=208352|stat=I|comm=kworker/1:1-events|reason=Idle kernel thread | note=Dry-run - not touched

292dryrun_unusedpid=209729|stat=I|comm=kworker/u8:0-flush-259:0|reason=Idle kernel thread | note=Dry-run - not touched

293dryrun_unusedpid=210380|stat=I|comm=kworker/1:3-cgroup_free|reason=Idle kernel thread | note=Dry-run - not touched

294dryrun_unusedpid=211798|stat=I|comm=kworker/u8:2-ext4-rsv-conversion|reason=Idle kernel thread | note=Dry-run - not touched

295docker_volumescount=4|reason=skipped

Unprocessed report -> /home/ubuntu/logs/prod_cleanup/unprocessed_20260820T172908Z.json

Cleanup Completed!

/home/ubuntu/./metric.py:1045: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.fromtimestamp(timestamp, datetime.UTC).
start_dt = datetime.datetime.utcnow().fromtimestamp(SCRIPT_START)
/home/ubuntu/./metric.py:1046: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
end_dt = datetime.datetime.utcnow()

Run Duration

Started: 2026-08-20 17:29:08 UTC
Finished: 2026-08-20 17:33:08 UTC
Duration: 3m 59s (239.9s total)

Section Timing Summary

Section	Time	% of Total
System Monitor	0.0s	0%
Top Processes	0.1s	0%
EBS Volumes	1.2s	1%
Unused Process Detection	6.1s	3%
Docker Cleanup	32.3s	13%
Disk Usage	33.7s	14%
TOTAL	239.9s	

Audit Logs

Log folder: /home/ubuntu/logs/prod_cleanup

cleanup_20260820T172908Z.ndjson - structured run events (NDJSON)
audit_master.ndjson - master append-only audit trail
deletions_20260820T172908Z.json - deleted items with restore hints
unprocessed_20260820T172908Z.json - skipped / failed items

```
#!/usr/bin/env python3
```

```
"""
```

Production Cleanup & Monitoring Script — Enhanced v2.0

Fixes: Dynamic file/directory deletion (IsADirectoryError resolved)

New: Docker mode selector, live runtime clock, ASCII bar graphs,
restoration log, unprocessed items tracker, unified audit log folder,
Unused Process Detection mode selector (independent of global mode)

```
"""
```

```
import os
import json
import shutil
import socket
import subprocess
import datetime
import getpass
import grp
import pwd
import time
import platform
import signal
import sys
from pathlib import Path
from rich.console import Console
from rich.prompt import Prompt, Confirm
from rich.table import Table
from rich.panel import Panel
from rich.live import Live
from rich.text import Text
from rich.columns import Columns
from rich import box
```

```
console = Console()
```

```
# ——— Unified log folder (best practice: one dir, rotate-friendly) ———
```

```
LOG_ROOT_CANDIDATES = [Path("/var/log/prod_cleanup"), Path.home() / "logs" / "prod_cleanup"]
```

```
LOG_DIR: Path = next((p for p in LOG_ROOT_CANDIDATES
                      if (p.exists() and os.access(p, os.W_OK))
                      or (not p.exists() and os.access(p.parent, os.W_OK))), LOG_ROOT_CANDIDATES[1])
```

```
RUN_TS = datetime.datetime.utcnow().strftime("%Y%m%dT%H%M%S")
```

```
LOG_FILE = LOG_DIR / f"cleanup_{RUN_TS}.ndjson" # per-run structured log
```

```
AUDIT_FILE = LOG_DIR / "audit_master.ndjson" # append-only master audit
```

```
DELETION_LOG = LOG_DIR / f"deletions_{RUN_TS}.json" # what was deleted + restore hints
```

```
UNPROCESSED = LOG_DIR / f"unprocessed_{RUN_TS}.json" # items user skipped / failed
```

— Global timing

```
SCRIPT_START = time.time()
SECTION_TIMINGS: dict = {}
```

— Unprocessed + deletion tracker

```
UNPROCESSED_ITEMS: list = []
DELETED_ITEMS: list = []
```

— Global state

—

```
RUNNER_IDENTITY: dict = {}
DOCKER_MODE: str = "2" # 1=Auto, 2=Manual — set during startup
```

#

HELPERS

#

```
def elapsed_since(start: float) -> str:
    secs = time.time() - start
    if secs < 60:
        return f"{secs:.1f}s"
    mins, s = divmod(int(secs), 60)
    return f"{mins}m {s}s"
```

```
def runtime_str() -> str:
    """Human-friendly total elapsed since script start."""
    return elapsed_since(SCRIPT_START)
```

```
def now_utc() -> str:
    return datetime.datetime.utcnow().strftime("%Y-%m-%dT%H:%M:%SZ")
```

```
def section_timer(name: str, start: float):
    elapsed = time.time() - start
    SECTION_TIMINGS[name] = elapsed
    console.print(f"[dim]⦿ {name} completed in {elapsed:.1f}s (total runtime: {runtime_str()}[/dim])")
```



```
def run_cmd(cmd, timeout=30) -> str:
    try:
        return subprocess.check_output(cmd, shell=True, text=True,
                                       stderr=subprocess.DEVNULL, timeout=timeout)
    except Exception:
        return ""
```

```
def docker_available() -> bool:
    return bool(run_cmd("which docker").strip())
```

```
#
```

```
# LOG SETUP — ensures log folder exists with README
```

```
#
```

```
def init_log_dir():
    LOG_DIR.mkdir(parents=True, exist_ok=True)
    readme = LOG_DIR / "README.txt"
    if not readme.exists():
        readme.write_text(
            "Production Cleanup Script — Audit Log Folder\n"
            "=====\n"
            "cleanup_*.ndjson → Per-run structured event log (one JSON per line)\n"
            "audit_master.ndjson → Append-only master audit trail (all runs)\n"
            "deletions_*.json → Files/dirs deleted + size + restore hints\n"
            "unprocessed_*.json → Items that were skipped or failed\n"
        )
```

```
def log_event(event: str, detail: str, extra: dict | None = None):
```

```
    ri = RUNNER_IDENTITY
    geo = ri.get("geo", {})
    lu = ri.get("linux_user", {})
    aws = ri.get("aws", {})
    ssh = ri.get("session", {})
```

```
    record = {
        "ts": now_utc(),
        "run_id": RUN_TS,
        "event": event,
        "detail": detail,
        "runtime_secs": round(time.time() - SCRIPT_START, 2),
        "hostname": ri.get("hostname", socket.gethostname()),
```

```

"local_ip": ri.get("local_ip", ""),
"public_ip": ri.get("public_ip", ""),
"geo_country": geo.get("country", ""),
"geo_city": geo.get("city", ""),
"linux_user": lu.get("username", getpass.getuser()),
"uid": lu.get("uid", os.getuid()),
"is_root": lu.get("is_root", os.geteuid() == 0),
"sudo_user": ssh.get("sudo_user", ""),
"aws_instance": aws.get("instance_id", ""),
"aws_region": aws.get("region", ""),
"aws_arn": aws.get("aws_arn", ""),
}
if extra:
    record.update(extra)

```

```

LOG_DIR.mkdir(parents=True, exist_ok=True)
line = json.dumps(record) + "\n"
LOG_FILE.open("a").write(line)
AUDIT_FILE.open("a").write(line)

```

```
#
```

```
# ASCII BAR GRAPHS
```

```
#
```

```

def _bar(pct: float, width: int = 30, color: str = "green") -> str:
    """Return a rich-markup ASCII progress bar for `pct` (0–100)."""
    pct = max(0.0, min(100.0, pct))
    filled = int(round(pct / 100 * width))
    bar_color = "green" if pct < 60 else ("yellow" if pct < 85 else "red")
    bar = f"[{bar_color}]{█" * filled}[/{bar_color}]{░ * (width - filled)}"
    return f"{bar} [bold]{pct:5.1f}%[/bold]"

```

```

def show_resource_graphs():
    """Print ASCII bar-graph dashboard for CPU, RAM, Disk, load average."""
    console.print("\n[bold cyan]===== RESOURCE USAGE GRAPHS =====[/bold cyan]")

```

```

def _cpu_pct() -> list[tuple[str, float]]:
    lines1 = Path("/proc/stat").read_text().splitlines()
    time.sleep(0.4)
    lines2 = Path("/proc/stat").read_text().splitlines()
    result = []
    for l1, l2 in zip(lines1, lines2):

```

```

    if not l1.startswith("cpu"):
        continue
    p1 = list(map(int, l1.split()[1:]))
    p2 = list(map(int, l2.split()[1:]))
    idle1, idle2 = p1[3], p2[3]
    tot1, tot2 = sum(p1), sum(p2)
    dtot = tot2 - tot1
    if dtot == 0:
        continue
    pct = 100.0 * (1 - (idle2 - idle1) / dtot)
    result.append((l1.split()[0], pct))
return result

cpu_data = _cpu_pct()

mem_info = {}
for line in Path("/proc/meminfo").read_text().splitlines():
    k, v = line.split(":")[0], line.split(":")[1].strip().split()[0]
    mem_info[k] = int(v)
mem_total = mem_info.get("MemTotal", 1)
mem_avail = mem_info.get("MemAvailable", mem_total)
mem_used = mem_total - mem_avail
mem_pct = 100.0 * mem_used / mem_total
swap_total = mem_info.get("SwapTotal", 0)
swap_free = mem_info.get("SwapFree", swap_total)
swap_used = swap_total - swap_free
swap_pct = 100.0 * swap_used / swap_total if swap_total else 0.0

def _kb(kb: int) -> str:
    if kb > 1_048_576:
        return f"{kb/1_048_576:.1f} GB"
    return f"{kb/1024:.0f} MB"

disk_data = []
for line in run_cmd("df -h --output=target,pcent,used,avail,size").splitlines()[1:]:
    parts = line.split()
    if len(parts) >= 5 and parts[0].startswith("/"):
        mnt = parts[0]
        try:
            pct = float(parts[1].rstrip("%"))
        except ValueError:
            continue
        disk_data.append((mnt, pct, parts[2], parts[3], parts[4]))

load_raw = Path("/proc/loadavg").read_text().split()
load1, load5, load15 = float(load_raw[0]), float(load_raw[1]), float(load_raw[2])
cores = os.cpu_count() or 1
load_pct1 = min(100.0, load1 / cores * 100)

```

```

console.print(Panel(
    f"[bold]Script started:[/bold]"
{datetime.datetime.utcnow().strftime('%Y-%m-%d %H:%M:%S')} UTC\n"
    f"[bold]Current time: [/bold] {datetime.datetime.utcnow().strftime('%Y-%m-%d %H:%M:%S')}
UTC\n"
    f"[bold]Elapsed:    [/bold] [yellow]{runtime_str()}[/yellow]",
    title="🕒 Runtime Clock", border_style="blue", padding=(0, 2)
))

```

```

console.print("\n[bold yellow] CPU[/bold yellow]")
for label, pct in cpu_data[:9]:
    tag = "[bold]ALL[/bold]" if label == "cpu" else label
    console.print(f" {tag:>6} {_bar(pct)}")

```

```

console.print(f"\n[bold yellow] Load Average[/bold yellow] "
    f"(cores={cores}): "
    f"[cyan]1m={load1}[/cyan] [cyan]5m={load5}[/cyan] [cyan]15m={load15}[/cyan]")
console.print(f" {'load':>6} {_bar(load_pct1)}")

```

```

console.print(f"\n[bold yellow] Memory[/bold yellow]")
console.print(f" {'RAM':>6} {_bar(mem_pct)} {_kb(mem_used)} / {_kb(mem_total)}")
if swap_total:
    console.print(f" {'Swap':>6} {_bar(swap_pct)} {_kb(swap_used)} / {_kb(swap_total)}")

```

```

console.print(f"\n[bold yellow] Disk Partitions[/bold yellow]")
for mnt, pct, used, avail, size in disk_data[:8]:
    console.print(f" {mnt:>20} {_bar(pct)} {used} used / {size} ({avail} free)")

```

```

log_event("graphs", "Resource graphs displayed",
    {"load1": load1, "mem_pct": round(mem_pct, 1)})

```

```
#
```

```
# RUNNER IDENTITY
```

```
#
```

```
def _fetch_imdsv2(path: str, token: str, timeout: int = 2) -> str:
```

```
    try:
```

```
        return subprocess.check_output(
            ["curl", "-sf", "-H", f"X-aws-ec2-metadata-token: {token}",
            f"http://169.254.169.254/latest/meta-data/{path}"],
            text=True, timeout=timeout).strip()
    except Exception:
```

```
return "unavailable"
```

```
def _get_imdsv2_token(timeout: int = 2) -> str:
```

```
    try:
```

```
        return subprocess.check_output(
            ["curl", "-sf", "-X", "PUT",
             "-H", "X-aws-ec2-metadata-token-ttl-seconds: 60",
             "http://169.254.169.254/latest/api/token"],
            text=True, timeout=timeout).strip()
```

```
    except Exception:
```

```
        return ""
```

```
def _get_public_ip() -> dict:
```

```
    try:
```

```
        raw = subprocess.check_output(
            ["curl", "-sf", "--max-time", "4",
             "http://ip-api.com/json?fields=status,message,country,countryCode,"
             "region,regionName,city,zip,lat,lon,timezone,isp,org,as,query"],
            text=True, timeout=5).strip()
```

```
        data = json.loads(raw)
```

```
        if data.get("status") == "success":
```

```
            return data
```

```
    except Exception:
```

```
        pass
```

```
    return {"query": "unavailable", "country": "unknown", "city": "unknown",
            "regionName": "unknown", "isp": "unknown", "timezone": "unknown"}
```

```
def _get_local_ip() -> str:
```

```
    try:
```

```
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

```
        s.connect(("8.8.8.8", 80))
```

```
        ip = s.getsockname()[0]; s.close(); return ip
```

```
    except Exception:
```

```
        return "unavailable"
```

```
def _get_linux_user_data() -> dict:
```

```
    try:
```

```
        pw = pwd.getpwuid(os.getuid())
```

```
        all_groups = [{"gid": g.gr_gid, "name": g.gr_name}
```

```
                       for g in grp.getgrall()]
```

```
                       if pw.pw_name in g.gr_mem or g.gr_gid == pw.pw_gid]
```

```
    return {"username": pw.pw_name, "uid": pw.pw_uid, "gid": pw.pw_gid,
            "gecos": pw.pw_gecos, "home_dir": pw.pw_dir, "shell": pw.pw_shell,
            "groups": all_groups, "euid": os.geteuid(), "egid": os.getegid(),
```

```

        "is_root": os.geteuid() == 0}
except Exception as e:
    return {"error": str(e)}

def _get_ssh_session() -> dict:
    return {k: os.environ.get(v, "") for k, v in [
        ("ssh_client", "SSH_CLIENT"), ("ssh_connection", "SSH_CONNECTION"),
        ("ssh_tty", "SSH_TTY"), ("term", "TERM"), ("tmux", "TMUX"),
        ("sudo_user", "SUDO_USER"), ("sudo_command", "SUDO_COMMAND"), ("logname", "LOGNAME")
    ]}

def _get_aws_identity() -> dict:
    result = {}
    token = _get_imds_v2_token()
    fetch = (lambda p: _fetch_imds_v2(p, token)) if token else (lambda p: "unavailable")
    for key, path in [("instance_id", "instance-id"), ("instance_type", "instance-type"),
        ("ami_id", "ami-id"), ("local_ipv4", "local-ipv4"),
        ("availability_zone", "placement/availability-zone"),
        ("region", "placement/region")]:
        result[key] = fetch(path)
    iam_role = fetch("iam/security-credentials/")
    result["iam_role"] = iam_role if iam_role and "unavailable" not in iam_role else "none"
    try:
        sts = json.loads(subprocess.check_output(
            ["aws", "sts", "get-caller-identity", "--output", "json"],
            text=True, timeout=5, stderr=subprocess.DEVNULL))
        result.update({"aws_account_id": sts.get("Account", ""), "aws_arn": sts.get("Arn", ""),
            "aws_user_id": sts.get("UserId", "")})
    except Exception:
        result.update({"aws_account_id": "unavailable", "aws_arn": "unavailable",
            "aws_user_id": "unavailable"})
    return result

def collect_runner_identity() -> dict:
    console.print("[dim] Collecting runner identity...[/dim]", end="")
    geo = _get_public_ip()
    identity = {
        "script_start_utc": now_utc(),
        "hostname": socket.gethostname(), "fqdn": socket.getfqdn(),
        "os": platform.platform(), "kernel": platform.release(),
        "python": platform.python_version(),
        "local_ip": _get_local_ip(), "public_ip": geo.get("query", "unavailable"),
        "geo": {"country": geo.get("country", ""), "country_code": geo.get("countryCode", ""),
            "region": geo.get("regionName", ""), "city": geo.get("city", ""),
            "latitude": geo.get("lat"), "longitude": geo.get("lon"),

```

```

        "timezone": geo.get("timezone", ""), "isp": geo.get("isp", ""),
        "org": geo.get("org", ""), "as_number": geo.get("as", ""),
        "linux_user": _get_linux_user_data(),
        "session": _get_ssh_session(),
        "aws": _get_aws_identity(),
    }
    console.print(" [green]done[/green]")
    return identity

```

```

def show_runner_identity():
    ri = RUNNER_IDENTITY
    geo = ri.get("geo", {}); lu = ri.get("linux_user", {})
    aws = ri.get("aws", {}); ssh = ri.get("session", {})
    groups_str = ", ".join(f"{g['name']}({g['gid']})" for g in lu.get("groups", []))
    ssh_src = ""
    if ssh.get("ssh_client"):
        parts = ssh["ssh_client"].split()
        ssh_src = f"{parts[0]}:{parts[1]}" if len(parts) >= 2 else ssh["ssh_client"]
    loc_str = ", ".join(filter(None, [geo.get("city"), geo.get("region"), geo.get("country")])) or "unknown"

    lines = [
        f"[bold white]Hostname:[/bold white]    {ri.get('hostname')} ({ri.get('fqdn')})",
        f"[bold white]OS/Kernel:[/bold white]    {ri.get('os')} — {ri.get('kernel')}",
        f"[bold white]Started:[/bold white]    {ri.get('script_start_utc')} | Elapsed:
[yellow]{runtime_str()}[/yellow]",
        "", f"[bold cyan]— Network —————[/bold cyan]",
        f"[bold white]Local IP:[/bold white]    {ri.get('local_ip')}",
        f"[bold white]Public IP:[/bold white]    {ri.get('public_ip')}",
        f"[bold white]Location:[/bold white]    {loc_str} | TZ: {geo.get('timezone')}",
        f"[bold white]ISP/Org:[/bold white]    {geo.get('isp')} / {geo.get('org')}",
        "", f"[bold cyan]— Linux User —————[/bold cyan]",
        f"[bold white]Username:[/bold white]    {lu.get('username')} (uid={lu.get('uid')},
euid={lu.get('euid')})",
        f"[bold white]Root?:[/bold white]    {'[red]YES — running as root![/red]' if lu.get('is_root') else
'[green]No[/green]'}",
        f"[bold white]Home/Shell:[/bold white]    {lu.get('home_dir')} | {lu.get('shell')}",
        f"[bold white]Groups:[/bold white]    {groups_str or '(none)'}",
        f"[bold white]SUDO_USER:[/bold white]    {ssh.get('sudo_user') or '(not sudo)'}",
        "", f"[bold cyan]— SSH Session —————[/bold cyan]",
        f"[bold white]SSH From:[/bold white]    {ssh_src or '(local / not SSH)'}",
        f"[bold white]TTY/TERM:[/bold white]    {ssh.get('ssh_tty') or '(none)'} | {ssh.get('term')}",
        f"[bold white]tmux:[/bold white]    {'yes' if ssh.get('tmux') else 'no'}",
        "", f"[bold cyan]— AWS Identity —————[/bold cyan]",
        f"[bold white]Instance:[/bold white]    {aws.get('instance_id')} ({aws.get('instance_type')})",
        f"[bold white]Region/AZ:[/bold white]    {aws.get('region')} / {aws.get('availability_zone')}",
        f"[bold white]Account:[/bold white]    {aws.get('aws_account_id')}",
        f"[bold white]IAM Role:[/bold white]    {aws.get('iam_role')}",

```

```

f"[bold white]STS ARN:[/bold white]    {aws.get('aws_arn'))",
"", f"[bold cyan]—— Log Folder ——[/bold cyan]",
f"[bold white]Log Dir:[/bold white]    {LOG_DIR}",
f"[bold white]Run Log:[/bold white]    {LOG_FILE.name}",
f"[bold white]Audit: [/bold white]    {AUDIT_FILE.name}",
f"[bold white]Deletions:[/bold white]    {DELETION_LOG.name}",
]
console.print(Panel("\n".join(lines),
    title="[bold yellow]🔍 RUNNER IDENTITY — Who Is Running This Script?[/bold yellow]",
    border_style="yellow", padding=(1, 2)))

#
=====
=====
# DESCRIPTION
#
=====
=====

def show_description():
    console.print(Panel("""
[bold yellow]Script Modes:[/bold yellow]

[bold green]1. Auto Mode[/bold green] — runs all actions automatically (pre-approved lists only)
[bold green]2. Interactive Mode [Default][[/bold green] — pauses and asks before every critical action

[bold cyan]Features v2.0:[/bold cyan]
• [magenta]Dynamic Deletion[/magenta] — files AND directories (recursive), safe + logged
• [magenta]Docker Mode Selector[/magenta] — choose Auto or Manual Docker cleanup at startup
• [magenta]Unused Process Mode Selector[/magenta] — independent kill mode per run
(Auto/Manual/Dry-Run)
• [magenta]ASCII Resource Graphs[/magenta] — bar charts for CPU / RAM / Disk / Load
• [magenta]Live Runtime Clock[/magenta] — shows start time + elapsed at every step
• [magenta]Runner Identity Logging[/magenta] — IP, geo, IAM, Linux user, SSH session
• [magenta]Unused Process Detection[/magenta] — idle / zombie / sleeping processes
• [magenta]Unified Log Folder[/magenta] — per-run + master audit + deletion + unprocessed logs
• [magenta]Restoration Hints[/magenta] — deletion log records size & location for recovery
• [magenta]Unprocessed Tracker[/magenta] — final report of everything skipped or failed
""",
    title="[bold cyan]Production Cleanup & Monitoring v2.0[/bold cyan]",
    border_style="cyan"))

#
=====
=====
# SYSTEM MONITOR

```



```
#
```

```
def system_monitor():
    t = time.time()
    console.print("\n[bold cyan]===== SYSTEM MONITOR =====[/bold cyan]")
    load = run_cmd("uptime | awk -F'load average:' '{print $2}").strip()
    console.print(f"[bold yellow]Load Avg:[/bold yellow] {load}")
    console.print(run_cmd("free -h"))
    console.print(run_cmd("df -h | grep -E '^/dev|Filesystem'"))
    log_event("monitor", "System stats displayed")
    section_timer("System Monitor", t)
```

```
#
```

```
# TOP PROCESSES
```

```
#
```

```
def show_top_processes():
    t = time.time()
    console.print("\n[bold cyan]===== TOP 20 CPU PROCESSES =====[/bold cyan]")
    cpu_output = run_cmd("ps -eo pid,ppid,comm,%cpu,%mem --sort=-%cpu | head -n 21").splitlines()
    _render_ps_table(cpu_output, "CPU")
    console.print("\n[bold cyan]===== TOP 20 MEMORY PROCESSES =====[/bold cyan]")
    mem_output = run_cmd("ps -eo pid,ppid,comm,%cpu,%mem --sort=-%mem | head -n 21").splitlines()
    _render_ps_table(mem_output, "MEM")
    log_event("monitor", "Top CPU/memory processes displayed")
    section_timer("Top Processes", t)
    return cpu_output, mem_output
```

```
def _render_ps_table(lines, sort_col):
    table = Table(show_header=True, header_style="bold magenta", box=box.SIMPLE)
    for col in ("PID", "PPID", "Command", "%CPU", "%MEM"):
        table.add_column(col, justify="right" if col != "Command" else "left", overflow="fold")
    for line in lines[1:]:
        parts = line.split(None, 4)
        if len(parts) == 5:
            table.add_row(*parts)
    console.print(table)
```

```
def kill_processes(process_list, label):
```

```

pids = Prompt.ask(
    f"Enter PIDs to kill from [yellow]{label}[/yellow] list (space-separated, blank=skip)",
    default="")
if pids:
    for pid in pids.split():
        try:
            os.kill(int(pid), 9)
            console.print(f"[green] ✓ Killed PID {pid}[/green]")
            log_event("kill", f"Process {pid} killed from {label}")
        except Exception as e:
            console.print(f"[red] ✗ Failed to kill {pid}: {e}[/red]")
            UNPROCESSED_ITEMS.append({"type": "kill", "pid": pid, "error": str(e)})

```

#

UNUSED PROCESS MODE SELECTOR ← NEW: independent of global mode

#

```
def select_unused_process_mode() -> str:
```

```
    """
```

Ask the user to choose a kill mode specifically for unused/idle processes.
 Completely independent of the global mode — runs every time this section
 is reached, allowing fine-grained control without touching other sections.

Returns

```
    -----
```

```

"1" Auto    — kill all zombies immediately, log & skip the rest
"2" Manual  — interactive: choose by row, PID, or skip per process
"3" Dry-Run — detect and display only, no kills performed
    """

```

```

console.print(Panel(
    "[bold green]1. Auto[/bold green]    "
    "— kill all [red]zombie[/red] processes automatically; log sleeping/idle as unprocessed\n\n"
    "[bold yellow]2. Manual[/bold yellow]  "
    "— review each candidate: kill by row, enter PIDs, or skip individually\n\n"
    "[bold blue]3. Dry-Run[/bold blue]    "
    "— detect and [underline]display only[/underline]; no processes will be touched "
    "(safe for auditing)",
    title="[bold cyan]🔧 Unused Process Detection — Kill Mode[/bold cyan]",
    border_style="cyan",
    padding=(1, 3),
))
choice = Prompt.ask(
    " Select unused-process mode",

```

```

        choices=["1", "2", "3"],
        default="3",
    )
    labels = {"1": "Auto", "2": "Manual", "3": "Dry-Run"}
    console.print(
        f"[dim] ✓ Unused-process mode set to: "
        f"[bold]{labels[choice]}/[bold]/[dim]"
    )
    log_event("unused_proc_mode", f"Unused-process mode selected: {labels[choice]}",
             {"unused_proc_mode": choice})
    return choice

#
=====
# UNUSED PROCESS DETECTION
#
=====

def detect_unused_processes():
    """
    Detect idle/zombie/sleeping processes and handle them according to a
    dedicated kill mode chosen right before scanning — independent of global mode.
    """
    t = time.time()
    console.print("\n[bold cyan]===== UNUSED / IDLE PROCESS DETECTION =====[/bold cyan]")

    # — Pick the kill mode NOW, before scanning —————
    u_mode = select_unused_process_mode()

    raw = run_cmd("ps -eo pid,stat,etime,%cpu,%mem,comm --no-headers").splitlines()
    unused = []
    for line in raw:
        parts = line.split(None, 5)
        if len(parts) < 6:
            continue
        pid, stat, etime, cpu, mem, comm = parts
        try: cpu_f = float(cpu)
        except ValueError: cpu_f = 0.0
        is_zombie = stat.startswith("Z")
        is_long_sleep = stat.startswith("S") and cpu_f == 0.0 and _etime_minutes(etime) > 60
        is_idle = stat.startswith("I")
        if is_zombie or is_long_sleep or is_idle:
            reason = ("Zombie" if is_zombie
                     else "Idle kernel thread" if is_idle
                     else f"Sleeping {_etime_minutes(etime):.0f}m, 0% CPU")

```

```

        unused.append((pid, stat, etime, cpu, mem, comm, reason))

if not unused:
    console.print("[green] ✓ No unused/zombie/idle processes found.[/green]")
    log_event("unused_procs", "None found")
    section_timer("Unused Process Detection", t)
    return

# ——— Render results table (always shown, even in dry-run) ———
table = Table(
    title=f"Unuses / Idle Processes — Mode: [bold
cyan][['Auto','Manual','Dry-Run'][int(u_mode)]][/bold cyan]",
    show_header=True,
    header_style="bold red",
    box=box.SIMPLE,
)
for col in ("#", "PID", "Stat", "Elapsed", "%CPU", "%MEM", "Command", "Reason", "Action"):
    table.add_column(col, overflow="fold")

for i, (pid, stat, etime, cpu, mem, comm, reason) in enumerate(unused, 1):
    if u_mode == "3":
        # Dry-Run: show what WOULD happen
        action_hint = (
            "[yellow]Would kill (zombie)[/yellow]" if stat.startswith("Z")
            else "[dim]Would skip (non-zombie)[/dim]"
        )
    elif u_mode == "1":
        # Auto: label what will be done
        action_hint = (
            "[red]→ Kill[/red]" if stat.startswith("Z")
            else "[dim]→ Log & skip[/dim]"
        )
    else:
        # Manual: pending user decision
        action_hint = "[cyan]Pending[/cyan]"
    table.add_row(str(i), pid, stat, etime, cpu, mem, comm, reason, action_hint)

console.print(table)
console.print(f"[yellow] Found {len(unused)} unused/idle process(es).[/yellow]")
log_event("unused_procs", f"Found {len(unused)}",
    {"pids": [u[0] for u in unused], "mode": u_mode})

# ——— Dispatch by mode ———
if u_mode == "3":
    _dryrun_unused(unused)
elif u_mode == "1":
    _auto_kill_unused(unused)
else:
    _interactive_kill_unused(unused)

```

```
section_timer("Unused Process Detection", t)
```

```
def _dryrun_unused(unused: list):
```

```
    """Dry-Run: report only — no process is touched."""
```

```
    zombies = [u for u in unused if u[1].startswith("Z")]
```

```
    sleepers = [u for u in unused if u[1].startswith("S")]
```

```
    idles = [u for u in unused if u[1].startswith("I")]
```

```
    console.print(Panel(
```

```
        f"[bold]Dry-Run Summary[/bold] — no changes made\n\n"
```

```
        f" [red]Zombies[/red]      : {len(zombies):>4} "
```

```
        f"{'(would be killed in Auto mode)' if zombies else ''}\n"
```

```
        f" [yellow]Long sleepers[/yellow] : {len(sleepers):>4} "
```

```
        f"{'(would be logged as unprocessed)}\n"
```

```
        f" [dim]Idle threads[/dim] : {len(idles):>4} "
```

```
        f"{'(would be logged as unprocessed)}\n\n"
```

```
        f" Total detected : {len(unused)}",
```

```
        title="[bold blue]🔍 Dry-Run Result[/bold blue]",
```

```
        border_style="blue",
```

```
        padding=(0, 2),
```

```
    ))
```

```
    # Still record them so the unprocessed report reflects what was seen
```

```
    for pid, stat, etime, cpu, mem, comm, reason in unused:
```

```
        UNPROCESSED_ITEMS.append({
```

```
            "type": "dryrun_unused",
```

```
            "pid": pid,
```

```
            "stat": stat,
```

```
            "comm": comm,
```

```
            "reason": reason,
```

```
            "note": "Dry-run — not touched",
```

```
        })
```

```
    log_event("unused_procs_dryrun",
```

```
        f"Dry-run: {len(unused)} processes reported, none killed",
```

```
        {"zombies": len(zombies), "sleepers": len(sleepers), "idles": len(idles)})
```

```
def _etime_minutes(etime: str) -> float:
```

```
    try:
```

```
        days = 0
```

```
        if "-" in etime:
```

```
            d, etime = etime.split("-"); days = int(d)
```

```
        parts = list(map(int, etime.split(":")))
```

```
        h, m, s = (parts + [0, 0, 0])[:3] if len(parts) == 3 else (0, parts[0], parts[1]) if len(parts) == 2 else (0, 0,
```

```
0)
```

```
        return days * 1440 + h * 60 + m + s / 60
```

```
    except Exception:
```

```
        return 0.0
```

```

def _auto_kill_unused(unused):
    console.print("\n[bold red][Auto Mode] Killing zombie processes...[/bold red]")
    for pid, stat, etime, cpu, mem, comm, reason in unused:
        if stat.startswith("Z"):
            try:
                os.kill(int(pid), 9)
                console.print(f"[green] ✓ Killed zombie PID {pid} ({comm})[/green]")
                log_event("kill_unused_auto", f"Zombie PID {pid} ({comm}) killed")
            except Exception as e:
                console.print(f"[red] ✗ Could not kill {pid}: {e}[/red]")
                UNPROCESSED_ITEMS.append({"type": "kill_zombie", "pid": pid, "error": str(e)})
        else:
            console.print(f"[yellow] ⚠ Logging PID {pid} ({comm}) — {reason}[/yellow]")
            UNPROCESSED_ITEMS.append({"type": "skip_unused", "pid": pid, "reason": reason})

def _interactive_kill_unused(unused):
    console.print("\n[bold yellow][Manual Mode] Choose how to handle unused processes:[/bold yellow]")
    console.print(" [1] Kill all zombies automatically [2] Pick by row [3] Enter PIDs [4] Skip all")
    choice = Prompt.ask("Your choice", choices=["1", "2", "3", "4"], default="4")
    if choice == "1":
        _auto_kill_unused(unused)
    elif choice == "2":
        nums = Prompt.ask("Row numbers to kill (space-separated)", default="")
        for n in nums.split():
            idx = int(n) - 1
            if 0 <= idx < len(unused):
                pid, _, _, _, comm, _ = unused[idx]
                _kill_pid(pid, comm, "row")
    elif choice == "3":
        pids = Prompt.ask("PIDs to kill (space-separated)", default="")
        pid_map = {u[0]: u[5] for u in unused}
        for pid in pids.split():
            _kill_pid(pid, pid_map.get(pid, "?"), "manual")
    else:
        console.print("[dim] All unused processes skipped.[/dim]")
        UNPROCESSED_ITEMS.append({"type": "unused_procs", "count": len(unused), "reason": "user skipped all"})

def _kill_pid(pid, comm, source):
    try:
        os.kill(int(pid), 9)
        console.print(f"[green] ✓ Killed PID {pid} ({comm})[/green]")
        log_event(f"kill_unused_{source}", f"PID {pid} ({comm})")

```

```
except Exception as e:
    console.print(f"[red] ✗ Failed to kill {pid}: {e}[/red]")
    UNPROCESSED_ITEMS.append({"type": "kill_fail", "pid": pid, "error": str(e)})
```

```
#
```

```
# EBS VOLUMES
```

```
#
```

```
def list_ebs_volumes():
    t = time.time()
    console.print(f"\n[bold cyan]===== EBS VOLUMES =====[/bold cyan]")
    if run_cmd("which aws"):
        out = run_cmd(
            'aws ec2 describe-volumes --query '
            '"Volumes[*].{ID:VolumeId,State:State,Size:Size,AZ:AvailabilityZone}" '
            '--output table | head -n 22')
        console.print(out if out else "[yellow]No volumes found[/yellow]")
        log_event("ebs", "Volumes listed")
    else:
        console.print("[yellow]AWS CLI not installed. Skipping.[/yellow]")
    section_timer("EBS Volumes", t)
```

```
#
```

```
# DISK USAGE
```

```
#
```

```
def disk_usage():
    t = time.time()
    path = Prompt.ask("Enter folder path to scan", default="/")
    console.print(f"\n[bold cyan]===== DISK USAGE ({path}) =====[/bold cyan]")
    usage = run_cmd(f"du -ah {path} 2>/dev/null | sort -rh | head -n 20")
    console.print(usage)
    log_event("disk", f"Top storage scanned in {path}")
    section_timer("Disk Usage", t)
```

```
#
```

```
# DYNAMIC DELETION ← FIXED: handles files AND directories
```

```
#
```

```
def _get_size(path: Path) -> int:
    """Return total size in bytes (works for both files and directories)."""
    try:
        if path.is_file() or path.is_symlink():
            return path.stat().st_size
        return sum(f.stat().st_size for f in path.rglob("*") if f.is_file())
    except Exception:
        return 0
```

```
def _human_size(size_bytes: int) -> str:
    for unit in ("B", "KB", "MB", "GB", "TB"):
        if size_bytes < 1024.0:
            return f"{size_bytes:.1f} {unit}"
        size_bytes /= 1024.0
    return f"{size_bytes:.1f} PB"
```

```
def _delete_path(target: str) -> bool:
    p = Path(target)
    if not p.exists() and not p.is_symlink():
        console.print(f"[yellow] ⚠ Not found: {target}[/yellow]")
        UNPROCESSED_ITEMS.append({"type": "delete_notfound", "path": target})
        return False
```

```
kind = "symlink" if p.is_symlink() else ("directory" if p.is_dir() else "file")
size_b = _get_size(p)
size_str = _human_size(size_b)
parent = str(p.parent)
name = p.name
```

```
if kind == "directory":
    console.print(f"[bold red] ⚠ {target} is a DIRECTORY ({size_str})[/bold red]")
    if not Confirm.ask(f" Recursively delete entire directory [red]{target}[/red]?", default=False):
        console.print(f"[yellow] ⚠ Skipped (directory deletion cancelled)[/yellow]")
        UNPROCESSED_ITEMS.append({"type": "delete_cancelled", "path": target, "kind": kind})
        return False
```

```
try:
    if kind == "directory":
```



```

        shutil.rmtree(target)
    else:
        os.remove(target)

    console.print(f"[green] ✓ Deleted {kind}: {target} ({size_str})[/green]")
    record = {
        "ts": now_utc(),
        "path": target,
        "kind": kind,
        "size_bytes": size_b,
        "size_human": size_str,
        "parent_dir": parent,
        "name": name,
        "restore_hint": f'Recover from backup at {parent}/{name} or check snapshots',
        "deleted_by": RUNNER_IDENTITY.get("linux_user", {}).get("username", getpass.getuser()),
        "run_id": RUN_TS,
    }
    DELETED_ITEMS.append(record)
    log_event("delete", f'Deleted {kind}: {target}',
             {"kind": kind, "size_bytes": size_b, "path": target})
    return True

except PermissionError as e:
    console.print(f"[red] ✗ Permission denied: {target} — {e}[/red]")
    UNPROCESSED_ITEMS.append({"type": "delete_error", "path": target, "error": str(e)})
    return False
except Exception as e:
    console.print(f"[red] ✗ Error deleting {target}: {e}[/red]")
    UNPROCESSED_ITEMS.append({"type": "delete_error", "path": target, "error": str(e)})
    return False

def delete_files():
    t = time.time()
    console.print("\n[bold cyan]===== DYNAMIC FILE / DIRECTORY DELETION =====[/bold cyan]")
    raw = Prompt.ask(
        "Enter full paths to delete (space-separated, blank=skip)",
        default="")
    if not raw.strip():
        console.print("[dim] Deletion skipped.[/dim]")
        return

    paths = raw.split()

    table = Table(title="Deletion Preview", show_header=True,
                  header_style="bold red", box=box.ROUNDED)
    table.add_column("#"); table.add_column("Path"); table.add_column("Type")
    table.add_column("Size", justify="right")

```

```

for i, p in enumerate(paths, 1):
    pp = Path(p)
    if not pp.exists() and not pp.is_symlink():
        table.add_row(str(i), p, "[yellow]NOT FOUND[/yellow]", "--")
    else:
        kind = "symlink" if pp.is_symlink() else ("directory" if pp.is_dir() else "file")
        table.add_row(str(i), p, kind, _human_size(_get_size(pp)))
console.print(table)

if not Confirm.ask(
    f"[bold red]Proceed to delete these {len(paths)} path(s)?[/bold red]", default=False):
    console.print("[dim] Deletion cancelled.[/dim]")
    UNPROCESSED_ITEMS.append({"type": "delete_batch_cancelled", "paths": paths})
    section_timer("Deletion", t)
    return

```

```

for p in paths:
    _delete_path(p)

```

```

DELETION_LOG.write_text(json.dumps({
    "run_id": RUN_TS,
    "deleted_at": now_utc(),
    "items": DELETED_ITEMS,
    "note": "Use restore_hint for recovery guidance."
}, indent=2))
console.print(f"\n[dim] Deletion log saved → {DELETION_LOG}[/dim]")
section_timer("Deletion", t)

```

```
#
```

```
# DOCKER CLEANUP — with its own mode selector
#
```

```

def select_docker_mode() -> str:
    console.print(Panel(
        "[bold green]1. Manual[/bold green] — review and confirm each Docker cleanup step\n"
        "[bold yellow]2. Auto[/bold yellow] — prune all stopped containers, dangling images, unused"
        "volumes automatically",
        title="[bold cyan]Docker Cleanup Mode[/bold cyan]",
        border_style="cyan", padding=(0, 2)))
    choice = Prompt.ask("Docker mode", choices=["1", "2"], default="1")
    log_event("docker_mode", f"Docker mode selected: {choice}")
    return choice

```

```

def docker_cleanup():
    t = time.time()
    console.print("\n[bold cyan]===== DOCKER CLEANUP =====[/bold cyan]")
    if not docker_available():
        console.print("[yellow] Docker not installed or not in PATH. Skipping.[/yellow]")
        log_event("docker", "Docker not available")
        section_timer("Docker Cleanup", t)
        return

    d_mode = select_docker_mode()
    _docker_stopped_containers(d_mode)
    _docker_dangling_images(d_mode)
    _docker_unused_volumes(d_mode)
    section_timer("Docker Cleanup", t)

def _docker_stopped_containers(mode: str):
    console.print("\n[bold yellow]— Stopped Containers —[/bold yellow]")
    raw = run_cmd(
        'docker ps -a --filter "status=exited" --filter "status=created" '
        '--format "{{.ID}}\t{{.Names}}\t{{.Image}}\t{{.Status}}\t{{.RunningFor}}"'
    ).strip()
    if not raw:
        console.print(" [green]✓ No stopped containers.[/green]"); return
    rows = [r.split("\t") for r in raw.splitlines() if r]
    table = Table(show_header=True, header_style="bold magenta", box=box.SIMPLE)
    for col in ("#", "Container ID", "Name", "Image", "Status", "Stopped For"):
        table.add_column(col, overflow="fold")
    for i, r in enumerate(rows, 1):
        table.add_row(str(i), *r)
    console.print(table)
    log_event("docker_containers", f"Found {len(rows)} stopped containers")
    if mode == "2":
        out = run_cmd("docker container prune -f")
        console.print(out)
        log_event("docker_containers_auto", "All stopped containers pruned")
    else:
        choice = Prompt.ask("[1] Remove all [2] Pick by row [3] Skip",
                           choices=["1", "2", "3"], default="3")
        if choice == "1":
            console.print(run_cmd("docker container prune -f"))
            log_event("docker_containers", "All pruned (manual)")
        elif choice == "2":
            nums = Prompt.ask("Row numbers (space-separated)", default="")
            for n in nums.split():
                idx = int(n) - 1
                if 0 <= idx < len(rows):
                    cid = rows[idx][0]

```

```

        out = run_cmd(f"docker rm -f {cid}")
        status = "[green]✓ [/green]" if out else "[red]✗ [/red]"
        console.print(f" {status} Removed container {cid}")
        log_event("docker_container_remove", f"Container {cid}")
    else:
        UNPROCESSED_ITEMS.append({"type": "docker_containers", "count": len(rows), "reason":
"skipped"})

def _docker_dangling_images(mode: str):
    console.print("\n[bold yellow]— Dangling Images —[/bold yellow]")
    raw = run_cmd(
        'docker images --filter "dangling=true" '
        '--format "{{.ID}}\t{{.Repository}}\t{{.Tag}}\t{{.Size}}\t{{.CreatedSince}}"'
    ).strip()
    if not raw:
        console.print(" [green]✓ No dangling images.[/green]"); return
    rows = [r.split("\t") for r in raw.splitlines() if r]
    table = Table(show_header=True, header_style="bold magenta", box=box.SIMPLE)
    for col in ("#", "Image ID", "Repository", "Tag", "Size", "Created"):
        table.add_column(col, overflow="fold")
    for i, r in enumerate(rows, 1):
        table.add_row(str(i), *r)
    console.print(table)
    log_event("docker_images", f"Found {len(rows)} dangling images")
    if mode == "2":
        console.print(run_cmd("docker image prune -f"))
        log_event("docker_images_auto", "All dangling images pruned")
    else:
        choice = Prompt.ask("[1] Remove all [2] Pick by row [3] Skip",
                             choices=["1", "2", "3"], default="3")
        if choice == "1":
            console.print(run_cmd("docker image prune -f"))
        elif choice == "2":
            nums = Prompt.ask("Row numbers (space-separated)", default="")
            for n in nums.split():
                idx = int(n) - 1
                if 0 <= idx < len(rows):
                    iid = rows[idx][0]
                    out = run_cmd(f"docker rmi -f {iid}")
                    console.print(f" {'[green]✓ [/green]'} if out else '[red]✗ [/red]'} Removed image {iid}")
        else:
            UNPROCESSED_ITEMS.append({"type": "docker_images", "count": len(rows), "reason":
"skipped"})

def _docker_unused_volumes(mode: str):
    console.print("\n[bold yellow]— Unused Volumes —[/bold yellow]")
    raw = run_cmd('docker volume ls --filter "dangling=true" --format "{{.Name}}\t{{.Driver}}"'
    ).strip()

```

```

if not raw:
    console.print(" [green]✓ No unused volumes.[/green]"); return
rows = [r.split("\t") for r in raw.splitlines() if r]
table = Table(show_header=True, header_style="bold magenta", box=box.SIMPLE)
for col in ("#", "Volume Name", "Driver"):
    table.add_column(col, overflow="fold")
for i, r in enumerate(rows, 1):
    table.add_row(str(i), *r)
console.print(table)
log_event("docker_volumes", f"Found {len(rows)} unused volumes")
if mode == "2":
    console.print(run_cmd("docker volume prune -f"))
    log_event("docker_volumes_auto", "All unused volumes pruned")
else:
    choice = Prompt.ask("[1] Remove all [2] Pick by row [3] Skip",
                        choices=["1", "2", "3"], default="3")
    if choice == "1":
        console.print(run_cmd("docker volume prune -f"))
    elif choice == "2":
        nums = Prompt.ask("Row numbers (space-separated)", default="")
        for n in nums.split():
            idx = int(n) - 1
            if 0 <= idx < len(rows):
                vname = rows[idx][0]
                out = run_cmd(f"docker volume rm {vname}")
                console.print(f" {'[green]✓[/green]'} if out else '[red]✗[/red]'} Removed volume {vname}")
            else:
                UNPROCESSED_ITEMS.append({"type": "docker_volumes", "count": len(rows), "reason":
"skipped"})

```

```
#
```

```
# UNPROCESSED ITEMS REPORT
```

```
#
```

```

def show_unprocessed_report():
    console.print("\n[bold cyan]===== UNPROCESSED / SKIPPED ITEMS =====[/bold cyan]")
    if not UNPROCESSED_ITEMS:
        console.print("[green] ✓ All items were processed successfully.[/green]")
    else:
        table = Table(title=f"⚠ {len(UNPROCESSED_ITEMS)} Unprocessed Item(s)",
                      show_header=True, header_style="bold yellow", box=box.ROUNDED)
        table.add_column("#"); table.add_column("Type"); table.add_column("Detail")
        for i, item in enumerate(UNPROCESSED_ITEMS, 1):

```

```

        detail = " | ".join(f"{k}={v}" for k, v in item.items() if k != "type")
        table.add_row(str(i), item.get("type", "unknown"), detail)
    console.print(table)

```

```

UNPROCESSED.parent.mkdir(parents=True, exist_ok=True)
UNPROCESSED.write_text(json.dumps({
    "run_id": RUN_TS, "ts": now_utc(),
    "count": len(UNPROCESSED_ITEMS),
    "items": UNPROCESSED_ITEMS
}, indent=2))
console.print(f"[dim] Unprocessed report → {UNPROCESSED}[/dim]")

```

```
#
```

```
# FINAL SUMMARY
```

```
#
```

```
def show_summary():
```

```

    total = time.time() - SCRIPT_START
    start_dt = datetime.datetime.utcfromtimestamp(SCRIPT_START)
    end_dt = datetime.datetime.utcnow()

```

```

console.print(Panel(
    f"[bold white]Started:[/bold white] {start_dt.strftime('%Y-%m-%d %H:%M:%S')} UTC\n"
    f"[bold white]Finished:[/bold white] {end_dt.strftime('%Y-%m-%d %H:%M:%S')} UTC\n"
    f"[bold white]Duration:[/bold white] [yellow]{runtime_str()}[/yellow] ({total:.1f}s total)",
    title="🕒 Run Duration", border_style="yellow", padding=(0, 2)))

```

```

table = Table(title="🕒 Section Timing Summary", show_header=True,
              header_style="bold cyan", box=box.ROUNDED)
table.add_column("Section", style="bold")
table.add_column("Time", justify="right")
table.add_column("% of Total", justify="right")
for section, secs in SECTION_TIMINGS.items():
    pct = secs / total * 100 if total else 0
    bar = "█" * int(pct / 5)
    table.add_row(section, f"{secs:.1f}s", f"{bar} {pct:.0f}%")
table.add_row("[bold yellow]TOTAL[/bold yellow]",
              f"[bold yellow]{total:.1f}s[/bold yellow]", "")
console.print(table)

```

```

console.print(Panel(
    f"[bold white]Log folder:[/bold white] {LOG_DIR}\n\n"
    f" [cyan]{LOG_FILE.name}[/cyan] — structured run events (NDJSON)\n"

```

```
f" [cyan]{AUDIT_FILE.name}[/cyan] — master append-only audit trail\n"
f" [cyan]{DELETION_LOG.name}[/cyan] — deleted items with restore hints\n"
f" [cyan]{UNPROCESSED.name}[/cyan] — skipped / failed items",
title="📁 Audit Logs", border_style="green", padding=(0, 2)))
```

```
#
```

```
# MAIN
```

```
#
```

```
def main():
    global RUNNER_IDENTITY
```

```
    init_log_dir()
    show_description()
```

```
    # — Runner identity
```

```
    console.print("\n[bold cyan]===== RUNNER IDENTITY =====[/bold cyan]")
    RUNNER_IDENTITY = collect_runner_identity()
    show_runner_identity()
    log_event("startup", "Script started — runner identity collected")
```

```
    # — Global mode selection
```

```
    mode = Prompt.ask(
        "\n[bold]Global Mode[/bold]: [green]1=Auto[/green] [yellow]2=Interactive[/yellow]",
        choices=["1", "2"], default="2")
    log_event("mode_selected", f"mode={mode}")
```

```
    # — Resource graphs
```

```
    show_resource_graphs()
```

```
    # — Standard sections
```

```
    system_monitor()
    cpu_list, mem_list = show_top_processes()
    list_ebs_volumes()
```

```
    # — Unused processes — own mode selector, independent of global mode —
    detect_unused_processes()      # mode chosen interactively inside
```

```
# — Docker — own mode selector
```

```
docker_cleanup()
```

```
# — Interactive-only
```

```
if mode == "2":  
    kill_processes(cpu_list, "CPU")  
    kill_processes(mem_list, "Memory")  
    disk_usage()  
    delete_files()
```

```
# — Final reports
```

```
show_unprocessed_report()  
log_event("finish", "Cleanup completed",  
    {"deleted_count": len(DELETED_ITEMS),  
     "unprocessed_count": len(UNPROCESSED_ITEMS)})
```

```
console.print("\n[bold green]  ✓ Cleanup Completed![/bold green]")  
show_summary()
```

```
if __name__ == "__main__":  
    main()
```


===== automation =====

=====

LINUX

=====

CPU — every 5 minutes

* /5 * * * * /usr/bin/python3 /opt/prod_cleanup.py --mode auto --linux cpu >>
/var/log/prod_cleanup.log 2>&1

Memory — every 5 minutes

* /5 * * * * /usr/bin/python3 /opt/prod_cleanup.py --mode auto --linux memory >>
/var/log/prod_cleanup.log 2>&1

Disk — daily at 2:00 AM

0 2 * * * /usr/bin/python3 /opt/prod_cleanup.py --mode auto --linux disk >>
/var/log/prod_cleanup.log 2>&1

Processes — daily at 3:00 AM

0 3 * * * /usr/bin/python3 /opt/prod_cleanup.py --mode auto --linux processes >>
/var/log/prod_cleanup.log 2>&1

EBS — daily at 4:00 AM

0 4 * * * /usr/bin/python3 /opt/prod_cleanup.py --mode auto --linux ebs >>
/var/log/prod_cleanup.log 2>&1

All Linux — every Sunday at 5:00 AM

0 5 * * 0 /usr/bin/python3 /opt/prod_cleanup.py --mode auto --linux all >>
/var/log/prod_cleanup.log 2>&1

```

# =====
# DOCKER
# =====

# Containers — daily at 6:00 AM
0 6 * * * /usr/bin/python3 /opt/prod_cleanup.py --mode auto --docker container-only >>
/var/log/prod_cleanup.log 2>&1

# Images — daily at 7:00 AM
0 7 * * * /usr/bin/python3 /opt/prod_cleanup.py --mode auto --docker image-only >>
/var/log/prod_cleanup.log 2>&1

# Volumes — every Sunday at 8:00 AM
0 8 * * 0 /usr/bin/python3 /opt/prod_cleanup.py --mode auto --docker volume-only >>
/var/log/prod_cleanup.log 2>&1

# All Docker — every Sunday at 9:00 AM
0 9 * * 0 /usr/bin/python3 /opt/prod_cleanup.py --mode auto --docker all >>
/var/log/prod_cleanup.log 2>&1


# =====
# EVERYTHING
# =====

# Linux + Docker — every Sunday at 10:00 AM
0 10 * * 0 /usr/bin/python3 /opt/prod_cleanup.py --mode auto --linux all --docker all >>
/var/log/prod_cleanup.log 2>&1


=====
# Linux
python3 prod_cleanup.py --mode auto --linux cpu
python3 prod_cleanup.py --mode auto --linux memory
python3 prod_cleanup.py --mode auto --linux disk
python3 prod_cleanup.py --mode auto --linux processes
python3 prod_cleanup.py --mode auto --linux ebs
python3 prod_cleanup.py --mode auto --linux all

# Docker
python3 prod_cleanup.py --mode auto --docker container-only
python3 prod_cleanup.py --mode auto --docker image-only
python3 prod_cleanup.py --mode auto --docker volume-only
python3 prod_cleanup.py --mode auto --docker all

```

Everything

python3 prod_cleanup.py --mode auto --linux all --docker all